

Министерство науки и высшего образования РФ
ФГАОУ ВПО
Национальный исследовательский технологический университет
«МИСИС»

Институт информационных технологий и компьютерных наук (ИТКН)
Кафедра инфокоммуникационных технологий (ИКТ)

Отчет по лабораторной работе № 8
по дисциплине «Объектно-ориентированное программирование»
на тему «Взаимодействие с СУБД в C#. Обработка XML-файлов»

Выполнил:
студент группы БИВТ-25-1
Кирюшин А. А.
Проверил:
доцент каф. ИКТ
Стучилин В.В.

Москва, 2026

1. ЦЕЛЬ РАБОТЫ

Закрепить навыки работы с базами данных и XML-документами на языке программирования C# в проекте с графическим интерфейсом на платформе Avalonia UI.

2. ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ

В рамках лабораторной работы были решены следующие задачи:

1. Разработать приложение с графическим интерфейсом для работы с RSS-лентой новостей (<https://lenta.ru/rss/news>) с использованием классов `HttpRequest` и `HttpResponse`
2. Вывести полный XML-текст загруженной ленты в текстовом окне приложения
3. Реализовать парсинг XML с использованием пространства имён `System.Xml` и вывести структурированную информацию: заголовок, аннотация, дата/время, ссылка
4. Использовать СУБД MySQL вместо SQLite для получения дополнительного балла (+4 балла)
5. Реализовать запись новостей в БД (с предварительным удалением старых записей) и чтение всех новостей из БД
6. Создать модель данных `NewsItem` с полями: `Id`, `Title`, `Description`, `Link`, `PubDate`, `CreatedAt`
7. Реализовать класс `RssService` для загрузки RSS-ленты по заданному URL
8. Реализовать класс `XmlParser` для парсинга RSS-ленты через `XmlDocument`
9. Реализовать класс `DatabaseManager` для работы с базой данных MySQL
10. Применить асинхронное программирование (`async/await`, `Task.Run`) для всех сетевых и БД операций
11. Реализовать обработку ошибок с кастомными диалоговыми окнами

3. СТРУКТУРА БАЗЫ ДАННЫХ MYSQL

Для хранения новостей создана база данных `rss_news` с таблицей `news`. Ниже представлена её структура.

Поле	Тип	Описание	Особенности
id	INT	Уникальный идентификатор	PRIMARY KEY, AUTO INCREMENT
title	VARCHAR(500)	Заголовок новости	NOT NULL

description	TEXT	Описание/аннотация	NULL допустим
link	VARCHAR(500)	Ссылка на полную новость	NULL допустим
pub_date	DATETIME	Дата публикации новости	Индексируется
created_at	TIMESTAMP	Время добавления в БД	DEFAULT CURRENT_TIMESTAMP

4. ЛИСТИНГ ПРОГРАММЫ

Класс `NewItem.cs`

Класс `NewItem` определяет модель данных новости с полями `Id`, `Title`, `Description`, `Link`, `PubDate` и `CreatedAt`.

```
using System;

namespace RssReaderApp.Models
{
    public class NewItem
    {
        public int Id { get; set; }
        public string Title { get; set; } = string.Empty;
        public string Description { get; set; } = string.Empty;
        public string Link { get; set; } = string.Empty;
        public DateTime PubDate { get; set; }
        public DateTime CreatedAt { get; set; }
    }
}
```

Класс `RssService.cs`

Класс `RssService` реализует загрузку RSS-ленты с веб-сервера с использованием классов `HttpWebRequest` и `HttpWebResponse`. Настраивает `UserAgent` и `Timeout` для корректной работы с серверами.

```
using System;
using System.IO;
using System.Net;

namespace RssReaderApp.Services
{
    public class RssService
    {
        private string rssUrl = "https://lenta.ru/rss/news";

        public void SetRssUrl(string url)
        {
            rssUrl = url;
        }

        public string DownloadRssFeed()
        {
            HttpWebRequest request =
                (HttpWebRequest) WebRequest.Create(rssUrl);
```

```

        request.UserAgent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
" +
        "AppleWebKit/537.36";
        request.Timeout = 30000;

        using (HttpWebResponse response =
(HttpWebResponse) request.GetResponse())
        using (Stream stream = response.GetResponseStream())
        using (StreamReader reader = new StreamReader(stream))
        {
            string rssContent = reader.ReadToEnd();
            return rssContent;
        }
    }
}

```

Класс XmlParser.cs

Класс XmlParser реализует парсинг RSS-ленты через пространство имён System.Xml. Использует XmlDocument для обработки XML, извлекает данные из тегов <item> и реализует надёжный парсинг дат в форматах RFC822 и RFC1123.

```

using System;
using System.Collections.Generic;
using System.Globalization;
using System.Xml;
using RssReaderApp.Models;

namespace RssReaderApp.Services
{
    public class XmlParser
    {
        public List<NewsItem> ParseRss(string xmlContent)
        {
            var news = new List<NewsItem>();
            try
            {
                XmlDocument doc = new XmlDocument();
                doc.LoadXml(xmlContent);

                XmlNodeList items = doc.GetElementsByTagName("item");
                foreach (XmlNode item in items)
                {
                    var newsItem = new NewsItem
                    {
                        Title = item["title"]?.InnerText ?? "",
                        Description = item["description"]?.InnerText ?? "",
                        Link = item["link"]?.InnerText ?? "",
                        PubDate =
ParsePubDate(item["pubDate"]?.InnerText ?? "")
                    };
                    news.Add(newsItem);
                }
            }
            catch { }
        }
    }
}

```

```

        }
    }
    catch (Exception ex)
    {
        throw new Exception($"Ошибка парсинга XML: {ex.Message}",
ex);
    }
    return news;
}

private DateTime ParsePubDate(string pubDate)
{
    if (string.IsNullOrEmpty(pubDate))
        return DateTime.Now;

    if (DateTime.TryParse(pubDate, CultureInfo.InvariantCulture,
        DateTimeStyles.None, out var date))
        return date;

    if (DateTime.TryParseExact(pubDate,
        "ddd, dd MMM yyyy HH:mm:ss zzz",
        CultureInfo.InvariantCulture,
        DateTimeStyles.None, out var date2))
        return date2;

    return DateTime.Now;
}
}
}

```

Класс DatabaseManager.cs

Класс DatabaseManager инкапсулирует логику работы с базой данных MySQL: создание таблицы, сохранение новостей (с удалением старых) и загрузку всех записей. Использует параметризованные запросы для защиты от SQL-инъекций.

```

using System;
using System.Collections.Generic;
using MySql.Data.MySqlClient;
using RssReaderApp.Models;

namespace RssReaderApp.Services
{
    public class DatabaseManager
    {
        private const string ConnectionString =
            "Server=localhost;Port=3306;Database=rss_news;" +
            "Uid=rss_user;Pwd=rss_pass;";

        public void InitializeDatabase()
        {
            using var connection = new MySqlConnection(ConnectionString);
            connection.Open();

```

```

        string createTableQuery = @"
            CREATE TABLE IF NOT EXISTS news (
                id            INT AUTO_INCREMENT PRIMARY KEY,
                title         VARCHAR(500) NOT NULL,
                description    TEXT,
                link          VARCHAR(500),
                pub_date      DATETIME,
                created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                INDEX idx_pub_date (pub_date)
            );

        using var command = new MySqlCommand(createTableQuery,
connection);
        command.ExecuteNonQuery();
    }

    public void SaveNews(List<NewsItem> newsList)
    {
        using var connection = new MySqlConnection(ConnectionString);
        connection.Open();

        using (var deleteCommand = new MySqlCommand("DELETE FROM news",
connection))
            deleteCommand.ExecuteNonQuery();

        foreach (var news in newsList)
        {
            string insertQuery = @"
                INSERT INTO news (title, description, link, pub_date)
                VALUES (@title, @description, @link, @pubDate)";

            using var cmd = new MySqlCommand(insertQuery, connection);
            cmd.Parameters.AddWithValue("@title",      news.Title);
            cmd.Parameters.AddWithValue("@description",
news.Description);
            cmd.Parameters.AddWithValue("@link",      news.Link);
            cmd.Parameters.AddWithValue("@pubDate",   news.PubDate);
            cmd.ExecuteNonQuery();
        }
    }

    public List<NewsItem> LoadAllNews()
    {
        var newsList = new List<NewsItem>();
        using var connection = new MySqlConnection(ConnectionString);
        connection.Open();

        string selectQuery = "SELECT * FROM news ORDER BY pub_date
DESC";

        using var cmd      = new MySqlCommand(selectQuery, connection);
        using var reader = cmd.ExecuteReader();

        while (reader.Read())

```

```

        {
            newsList.Add(new NewsItem
            {
                Id          = reader.GetInt32("id"),
                Title       = reader.GetString("title"),
                Description  =
reader.IsDBNull(reader.GetOrdinal("description"))
                    ? "" : reader.GetString("description"),
                Link        =
reader.IsDBNull(reader.GetOrdinal("link"))
                    ? "" : reader.GetString("link"),
                PubDate     = reader.GetDateTime("pub_date"),
                CreatedAt   = reader.GetDateTime("created_at")
            });
        }
        return newsList;
    }

    public bool TestConnection()
    {
        try
        {
            using var connection = new
MySQLConnection(ConnectionString);
            connection.Open();
            return true;
        }
        catch { return false; }
    }
}

```

Графический интерфейс (MainWindow.axaml)

XAML-разметка определяет структуру пользовательского интерфейса: поле ввода URL RSS-ленты, кнопки управления, окно отображения полного XML, список распарсенных новостей, кнопки работы с БД и окно вывода данных из базы.

```

<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:models="clr-namespace:RssReaderApp.Models"
        x:Class="RssReaderApp.Views.MainWindow"
        Title="RSS Reader - Лабораторная работа №8"
        Width="1200" Height="900">

    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="200"/>
            <RowDefinition Height="*/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="200"/>
        </Grid.RowDefinitions>
    </Grid>

```

```

</Grid.RowDefinitions>

<!-- Адрес RSS -->
<StackPanel Grid.Row="0" Margin="0,0,0,10">
    <TextBlock Text="Адрес RSS-ленты:" FontWeight="Bold"
Margin="0,0,0,5"/>
    <TextBox Name="RssUrlTextBox"
Text="https://lenta.ru/rss/news"/>
</StackPanel>

<!-- Кнопки RSS -->
<StackPanel Grid.Row="1" Orientation="Horizontal" Spacing="10"
Margin="0,0,0,10">
    <Button Name="DownloadRssButton" Content="Загрузить RSS"
Click="OnDownloadRss" Width="150" Height="35"
FontSize="14"/>
    <Button Name="ParseRssButton" Content="Распарсить XML"
Click="OnParseRss" Width="150" Height="35"
FontSize="14"/>
    <TextBlock Name="StatusTextBlock" VerticalAlignment="Center"
FontStyle="Italic" Foreground="Gray"
Margin="10,0,0,0"/>
</StackPanel>

<!-- Полный XML текст -->
<Border Grid.Row="2" BorderBrush="Gray" BorderThickness="1"
Margin="0,0,0,10">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
        </Grid.RowDefinitions>
        <TextBlock Grid.Row="0" Text="Полный текст RSS (XML):"
FontWeight="Bold" Margin="5"
Background="#F0F0F0"/>
        <ScrollView Grid.Row="1"
HorizontalScrollBarVisibility="Auto">
            <TextBox Name="RawXmlTextBox" IsReadOnly="True"
TextWrapping="Wrap" FontFamily="Consolas"
FontSize="11"/>
        </ScrollView>
    </Grid>
</Border>

<!-- Распарсенные новости -->
<Border Grid.Row="3" BorderBrush="Gray" BorderThickness="1"
Margin="0,0,0,10">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
        </Grid.RowDefinitions>
        <TextBlock Grid.Row="0" Text="Распарсенные новости:"
FontWeight="Bold" Margin="5"
Background="#F0F0F0"/>

```

```

        <ListBox Grid.Row="1" Name="ParsedNewsListBox" Padding="5">
            <ListBox.ItemTemplate>
                <DataTemplate DataType="models:NewsItem">
                    <Border BorderBrush="#E0E0E0"
BorderThickness="0,0,0,1"
                        Padding="10" Margin="0,0,0,5">
                            <StackPanel>
                                <TextBlock Text="{Binding Title}"
                                    FontWeight="Bold"
FontSize="14"
                                    TextWrapping="Wrap"
Margin="0,0,0,5"/>
                                <TextBlock Text="{Binding Description}"
                                    TextWrapping="Wrap"
Foreground="#555"/>
                                <StackPanel Orientation="Horizontal"
Spacing="15">
                                    <TextBlock FontSize="11"
Foreground="Gray"
                                        Text="{Binding PubDate,
                                            StringFormat='Дата:
{0:dd.MM.yyyy HH:mm}'}"/>
                                    <TextBlock Text="{Binding Link}"
                                        FontSize="11" Foreground="Blue"
                                        TextDecorations="Underline"/>
                                </StackPanel>
                            </StackPanel>
                        </Border>
                    </DataTemplate>
                </ListBox.ItemTemplate>
            </ListBox>
        </Grid>
    </Border>

    <!-- Кнопки БД -->
    <StackPanel Grid.Row="4" Orientation="Horizontal" Spacing="10"
Margin="0,0,0,10">
        <Button Name="SaveToDbButton" Content="Сохранить в БД"
            Click="OnSaveToDb" Width="150" Height="35"
FontSize="14"/>
        <Button Name="LoadFromDbButton" Content="Загрузить из БД"
            Click="OnLoadFromDb" Width="150" Height="35"
FontSize="14"/>
        <TextBlock Name="DbStatusTextBlock" VerticalAlignment="Center"
            FontStyle="Italic" Foreground="Gray"
Margin="10,0,0,0"/>
    </StackPanel>

    <!-- Новости из БД -->
    <Border Grid.Row="5" BorderBrush="Gray" BorderThickness="1">
        <Grid>
            <Grid.RowDefinitions>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="*/>
            </Grid.RowDefinitions>

```

```

        <TextBlock Grid.Row="0" Text="Новости из БД:"
                    FontWeight="Bold" Margin="5"
Background="#F0F0F0"/>
        <ScrollView Grid.Row="1">
            <TextBox Name="DbNewsTextBox" IsReadOnly="True"
                    TextWrapping="Wrap" FontFamily="Consolas"
FontSize="11"/>
        </ScrollView>
    </Grid>
</Border>
</Grid>
</Window>

```

Логика интерфейса (MainWindow.axaml.cs)

Code-behind файл содержит логику загрузки RSS, парсинга XML, взаимодействия с базой данных и отображения результатов. Все долгосрочные операции выполняются асинхронно через `async/await` и `Task.Run`.

```

using Avalonia.Controls;
using Avalonia.Interactivity;
using RssReaderApp.Models;
using RssReaderApp.Services;
using System;
using System.Collections.Generic;
using System.Text;
using System.Threading.Tasks;

namespace RssReaderApp.Views
{
    public partial class MainWindow : Window
    {
        private RssService rssService;
        private XmlParser xmlParser;
        private DatabaseManager dbManager;
        private string? currentXml;
        private List<NewsItem>? currentNews;

        public MainWindow()
        {
            InitializeComponent();
            rssService = new RssService();
            xmlParser = new XmlParser();
            dbManager = new DatabaseManager();
            CheckDatabaseConnection();
        }

        private void CheckDatabaseConnection()
        {
            try
            {
                if (dbManager.TestConnection())
                {

```

```

        dbManager.InitializeDatabase();
        DbStatusTextBlock.Text = "БД подключена";
        DbStatusTextBlock.Foreground =
Avalonia.Media.Brushes.Green;
    }
    else
    {
        DbStatusTextBlock.Text = "БД не подключена";
        DbStatusTextBlock.Foreground =
Avalonia.Media.Brushes.Red;
    }
}
catch (Exception ex)
{
    DbStatusTextBlock.Text = $"Ошибка БД: {ex.Message}";
    DbStatusTextBlock.Foreground = Avalonia.Media.Brushes.Red;
}
}

private async void OnDownloadRss(object? sender, RoutedEventArgs e)
{
    try
    {
        StatusTextBlock.Text = "Загрузка RSS...";
        StatusTextBlock.Foreground = Avalonia.Media.Brushes.Blue;
        DownloadRssButton.IsEnabled = false;

        rssService.SetRssUrl(RssUrlTextBox.Text ??
"https://lenta.ru/rss/news");
        currentXml = await Task.Run(() =>
rssService.DownloadRssFeed());
        RawXmlTextBox.Text = currentXml;

        StatusTextBlock.Text = "RSS загружен успешно";
        StatusTextBlock.Foreground = Avalonia.Media.Brushes.Green;
    }
    catch (Exception ex)
    {
        await ShowError("Ошибка загрузки RSS", ex.Message);
        StatusTextBlock.Text = $"Ошибка: {ex.Message}";
        StatusTextBlock.Foreground = Avalonia.Media.Brushes.Red;
    }
    finally { DownloadRssButton.IsEnabled = true; }
}

private async void OnParseRss(object? sender, RoutedEventArgs e)
{
    if (string.IsNullOrEmpty(currentXml))
    {
        await ShowError("Ошибка", "Сначала загрузите RSS");
        return;
    }
    try
    {
        StatusTextBlock.Text = "Парсинг XML...";
    }
}

```

```

        StatusTextBlock.Foreground = Avalonia.Media.Brushes.Blue;
        ParseRssButton.IsEnabled = false;

        currentNews = await Task.Run(() =>
xmlParser.ParseRss(currentXml));
        ParsedNewsListBox.ItemsSource = currentNews;

        StatusTextBlock.Text = $"Распарсено новостей:
{currentNews.Count}";
        StatusTextBlock.Foreground = Avalonia.Media.Brushes.Green;
    }
    catch (Exception ex)
    {
        await ShowError("Ошибка парсинга XML", ex.Message);
        StatusTextBlock.Text = $"Ошибка парсинга: {ex.Message}";
        StatusTextBlock.Foreground = Avalonia.Media.Brushes.Red;
    }
    finally { ParseRssButton.IsEnabled = true; }
}

private async void OnSaveToDb(object? sender, RoutedEventArgs e)
{
    if (currentNews == null || currentNews.Count == 0)
    {
        await ShowError("Ошибка", "Нет новостей для сохранения.");
        return;
    }
    try
    {
        DbStatusTextBlock.Text = "Сохранение в БД...";
        DbStatusTextBlock.Foreground = Avalonia.Media.Brushes.Blue;
        SaveToDbButton.IsEnabled = false;

        await Task.Run(() => dbManager.SaveNews(currentNews));

        DbStatusTextBlock.Text = $"Сохранено: {currentNews.Count}
новостей";
        DbStatusTextBlock.Foreground =
Avalonia.Media.Brushes.Green;
        await ShowInfo("Успех", $"Сохранено {currentNews.Count}
новостей в БД");
    }
    catch (Exception ex)
    {
        await ShowError("Ошибка сохранения в БД", ex.Message);
        DbStatusTextBlock.Foreground = Avalonia.Media.Brushes.Red;
    }
    finally { SaveToDbButton.IsEnabled = true; }
}

private async void OnLoadFromDb(object? sender, RoutedEventArgs e)
{
    try
    {
        DbStatusTextBlock.Text = "Загрузка из БД...";

```

```

        DbStatusTextBlock.Foreground = Avalonia.Media.Brushes.Blue;
        LoadFromDbButton.IsEnabled = false;

        var newsFromDb = await Task.Run(() =>
dbManager.LoadAllNews());
        var sb = new StringBuilder();
        sb.AppendLine($"=== ЗАГРУЖЕНО ИЗ БД: {newsFromDb.Count}
НОВОСТЕЙ ===");
        sb.AppendLine();
        foreach (var news in newsFromDb)
        {
            sb.AppendLine($"ID: {news.Id}");
            sb.AppendLine($"Заголовок: {news.Title}");
            sb.AppendLine($"Описание: {news.Description}");
            sb.AppendLine($"Ссылка: {news.Link}");
            sb.AppendLine($"Дата публикации:
{news.PubDate:dd.MM.yyyy HH:mm}");
            sb.AppendLine($"Добавлено в БД:
{news.CreatedAt:dd.MM.yyyy HH:mm:ss}");
            sb.AppendLine(new string('-', 60));
        }
        DbNewsTextBox.Text = sb.ToString();

        DbStatusTextBlock.Text = $"Загружено из БД:
{newsFromDb.Count} новостей";
        DbStatusTextBlock.Foreground =
Avalonia.Media.Brushes.Green;
    }
    catch (Exception ex)
    {
        await ShowError("Ошибка загрузки из БД", ex.Message);
        DbStatusTextBlock.Foreground = Avalonia.Media.Brushes.Red;
    }
    finally { LoadFromDbButton.IsEnabled = true; }
}

private async Task ShowError(string title, string message)
{
    var w = new Window { Title = title, Width = 400, Height = 200,
        WindowStartupLocation = WindowStartupLocation.CenterOwner,
        CanResize = false };
    var panel = new StackPanel { Margin = new
Avalonia.Thickness(20), Spacing = 15 };
    panel.Children.Add(new TextBlock { Text = message, FontSize =
14,
        TextWrapping = Avalonia.Media.TextWrapping.Wrap });
    var btn = new Button { Content = "OK", Width = 100, Height =
35,
        HorizontalAlignment =
Avalonia.Layout.HorizontalAlignment.Center };
    btn.Click += (s, e) => w.Close();
    panel.Children.Add(btn);
    w.Content = panel;
    await w.ShowDialog(this);
}

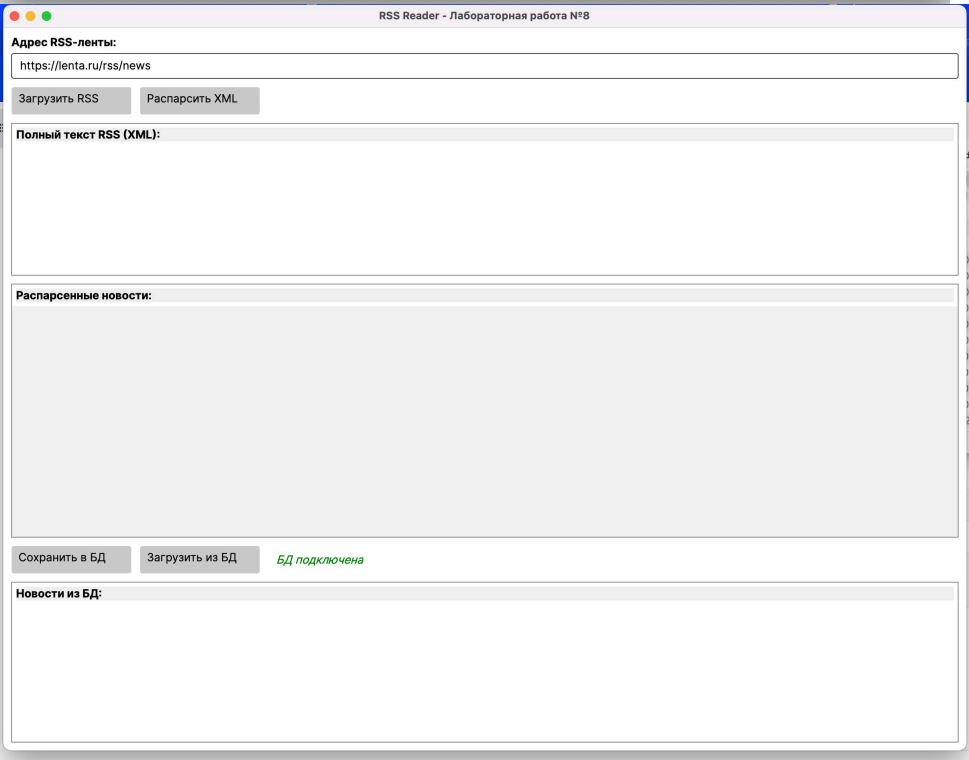
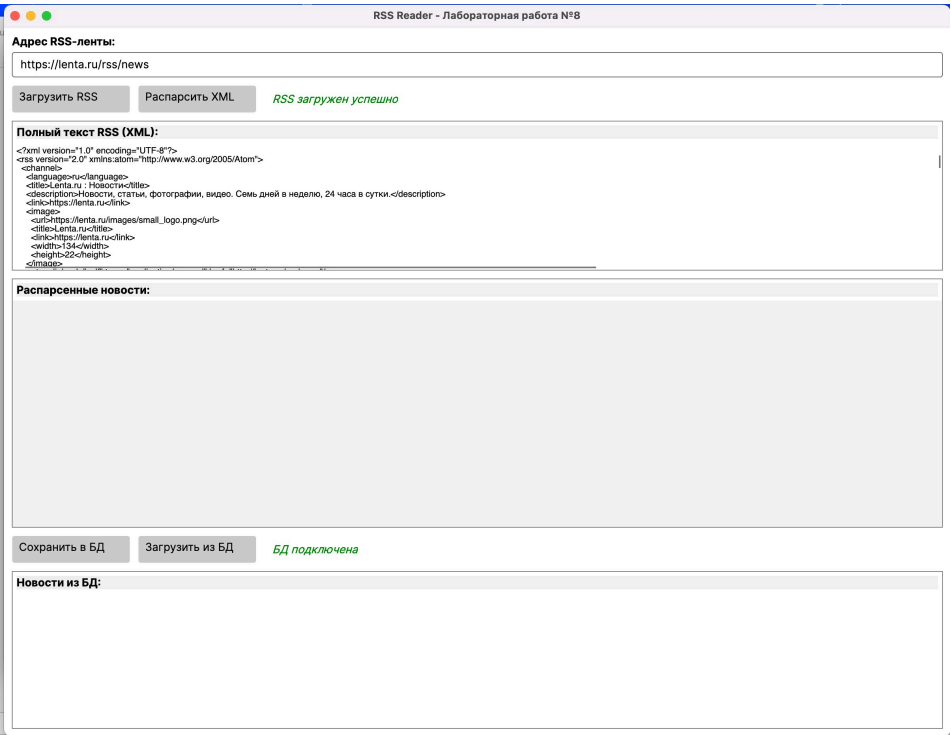
```

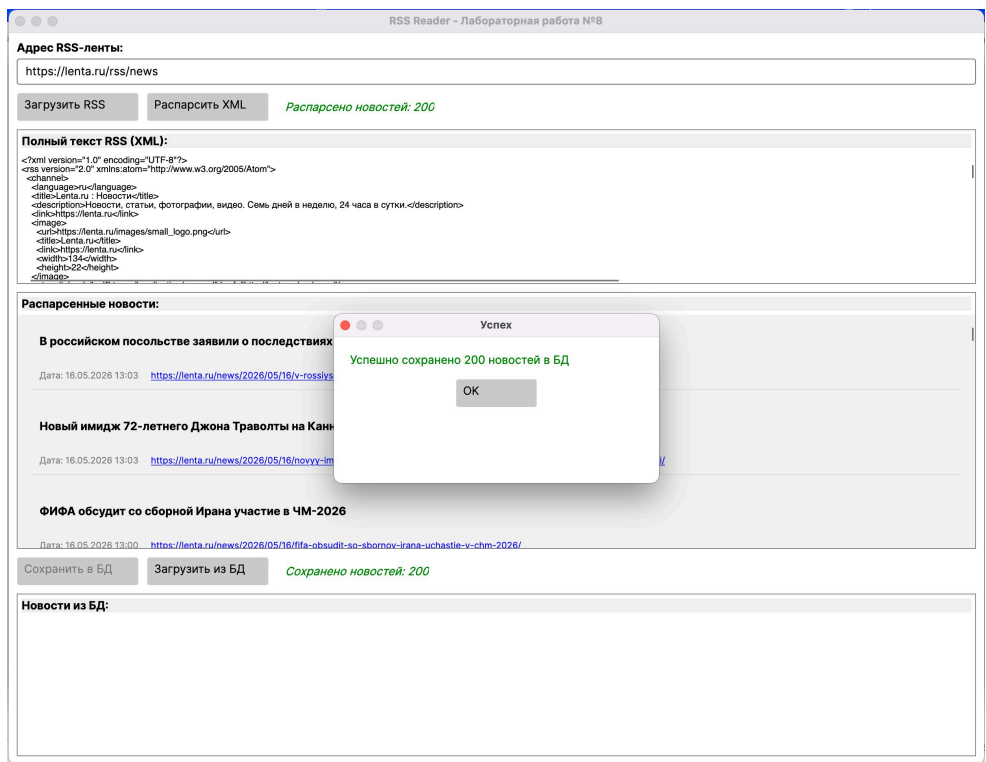
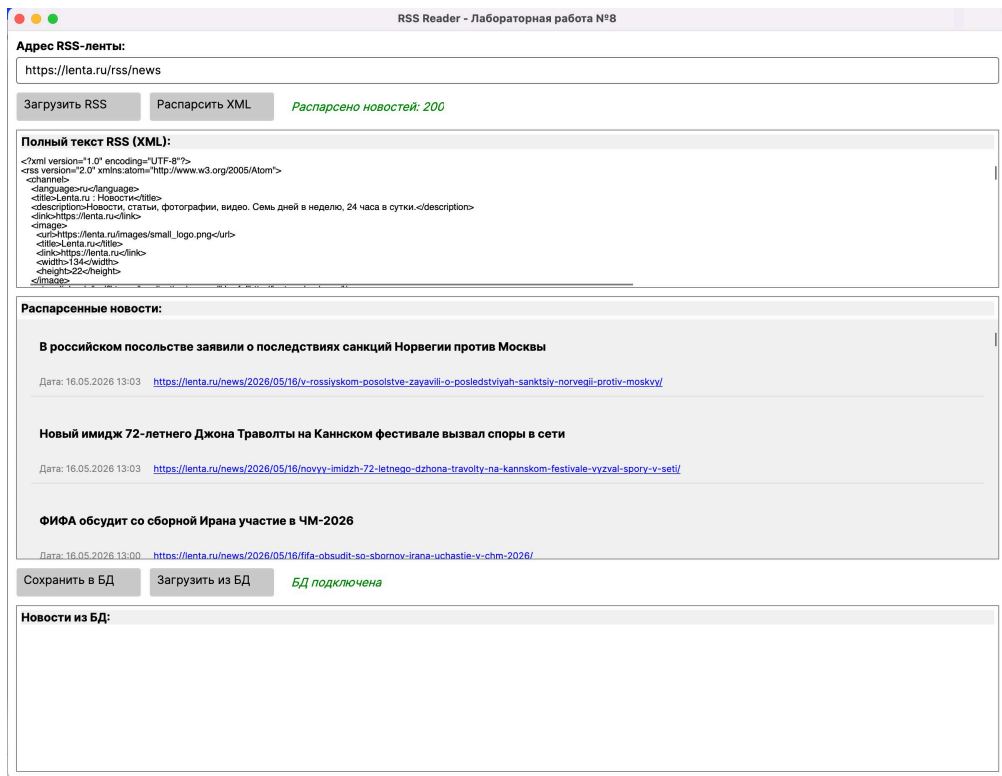
```

private async Task ShowInfo(string title, string message)
{
    var w = new Window { Title = title, Width = 400, Height = 180,
        WindowStartupLocation = WindowStartupLocation.CenterOwner,
        CanResize = false };
    var panel = new StackPanel { Margin = new
Avalonia.Thickness(20), Spacing = 15 };
    panel.Children.Add(new TextBlock { Text = message, FontSize =
14,
        TextWrapping = Avalonia.Media.TextWrapping.Wrap,
        Foreground = Avalonia.Media.Brushes.Green });
    var btn = new Button { Content = "OK", Width = 100, Height =
35,
        HorizontalAlignment =
Avalonia.Layout.HorizontalAlignment.Center };
    btn.Click += (s, e) => w.Close();
    panel.Children.Add(btn);
    w.Content = panel;
    await w.ShowDialog(this);
}
}
}

```

Выполнение:





RSS Reader - Лабораторная работа №8

Адрес RSS-ленты:

https://lenta.ru/rss/news

Загрузить RSS

Распарсить XML

Распарсено новостей: 200

Полный текст RSS (XML):

<?xml version="1.0" encoding="UTF-8"?>
<rss version="2.0" xmlns:atom="http://www.w3.org/2005/Atom">
<channel>
<language>ru</language>
<title>Lenta.ru : Новости</title>
<description>Новости, статьи, фотографии, видео. Семь дней в неделю, 24 часа в сутки.</description>
<link>https://lenta.ru</link>
<image>
<url>https://lenta.ru/images/small_logo.png</url>
<title>Lenta.ru</title>
<link>https://lenta.ru</link>
<width>134</width>
<height>22</height>
</image>

Распаршенные новости:

В российском посольстве заявили о последствиях санкций Норвегии против Москвы

Дата: 16.05.2026 13:03 <https://lenta.ru/news/2026/05/16/v-rossiyskom-posolstve-zayavili-o-posledstviyah-sanktsiy-norvegii-protiv-moskvy/>

Новый имидж 72-летнего Джона Траволты на Каннском фестивале вызвал споры в сети

Дата: 16.05.2026 13:03 <https://lenta.ru/news/2026/05/16/novyy-imidzh-72-letnego-dzhona-travolty-na-kannskom-festivale-vyzval-spory-v-seti/>

ФИФА обсудит со сборной Ирана участие в ЧМ-2026

Дата: 16.05.2026 13:00 <https://lenta.ru/news/2026/05/16/fifa-obsudit-so-sbornov-irana-uchastie-v-chm-2026/>

Сохранить в БД

Загрузить из БД

Загружено из БД: 200 новостей

Новости из БД:

=== ЗАГРУЖЕНО ИЗ БД: 200 НОВОСТЕЙ ===

ID: 401

Заголовок: В российском посольстве заявили о последствиях санкций Норвегии против Москвы

Описание:

Ссылка: <https://lenta.ru/news/2026/05/16/v-rossiyskom-posolstve-zayavili-o-posledstviyah-sanktsiy-norvegii-protiv-moskvy/>

Дата публикации: 16.05.2026 13:03

Добавлено в БД: 16.05.2026 13:09:12

ID: 402

Заголовок: Новый имидж 72-летнего Джона Траволты на Каннском фестивале вызвал споры в сети

Описание:

Ссылка: <https://lenta.ru/news/2026/05/16/novyy-imidzh-72-letnego-dzhona-travolty-na-kannskom-festivale-vyzval-spory-v-seti/>

Дата публикации: 16.05.2026 13:03

5. ВЫВОД

В ходе выполнения лабораторной работы я успешно освоил ключевые технологии для создания сетевых приложений с базами данных на платформе .NET. Получил практический опыт работы с классами `HttpRequest` и `HttpResponse` из пространства имён `System.Net` для загрузки данных с веб-серверов. Понял принципы работы HTTP-протокола, необходимость указания `UserAgent` для корректной работы с некоторыми серверами и важность настройки таймаутов. Реализовал загрузку RSS-ленты новостей с сайта `lenta.ru` и вывел полный XML-контент в текстовое окно приложения.

Освоил работу с XML-документами через пространство имён `System.Xml`. Применил класс `XmlDocument` для парсинга XML-структуры, использовал метод `GetElementsByTagName` для извлечения элементов `<item>` и навигацию через `XmlNode` для доступа к вложенным элементам. Реализовал надёжный парсинг дат в форматах RFC822 и RFC1123 с использованием `DateTime.TryParse` и `DateTime.TryParseExact`, что обеспечило корректную обработку временных меток из RSS-ленты.

Получил углублённые знания работы с реляционными базами данных MySQL на платформе C#. Использовал библиотеку `MySQL.Data` и класс `MySQLConnection` для установки соединения с сервером БД. Реализовал класс `DatabaseManager`, который инкапсулирует всю логику работы с базой данных: создание таблицы через `CREATE TABLE IF NOT EXISTS`, удаление старых записей через `DELETE`, вставку новых данных через параметризованные запросы `INSERT` с использованием `MySQLParameter` для защиты от SQL-инъекций, и чтение данных через `SELECT` с помощью `MySQLDataReader`.

Создал полнофункциональное графическое приложение на базе кроссплатформенного UI-фреймворка Avalonia. Разработал сложную XAML-разметку интерфейса с использованием `Grid` для деления окна на шесть функциональных зон. Применил продвинутые возможности `ListBox` с кастомным `DataTemplate` и явным указанием `DataType="models:NewsItem"` для красивого отображения структурированных новостей с заголовками, описаниями и датами публикации.

Освоил асинхронное программирование в C# с помощью ключевых слов `async` и `await`. Применил метод `Task.Run` для выполнения долгих операций (сетевые запросы, парсинг XML, операции с БД) в фоновом потоке, что обеспечило отзывчивость пользовательского интерфейса. Реализовал временное отключение кнопок через `IsEnabled = false` во время выполнения

операций для предотвращения повторных вызовов. Полученные навыки работы с сетевыми запросами, XML-парсингом, реляционными базами данных и асинхронным программированием являются фундаментальными для разработки современных приложений на платформе .NET.